

Modelling Everything

Abstract

In this paper we outline the model humans use to describe the world and formalize it. This will enable us to store raw knowledge in an unambiguous, defined format.

Things

Humans think in things. Everything we can think of is a thing. There is a set of things. The things themselves are just abstract; what is important is the semantics, the meaning of them. In everyday conversations, we as humans talk about things. And you can only talk about a thing if you can identify it. And if you cannot talk about a thing because you cannot identify it then it is not a thing because we could not think of it. Therefore there can only exist so many things as identifiers. In this paper we will use *object* as a synonym for thing.

Identifying Things

The straightforward way for us humans to talk about a thing are names. For example, we identify people by their names, places, activities, etc. The thing is (pun intended) that names can be ambiguous.

A person *R* growing up with banks as in river banks may ought to uniquely and unambiguously identify the concept of river banks with the word “bank”. A sales person *S* in the city may use the word “bank” to uniquely and unambiguously identify the bank as in where you can store money. A conversation of banks between may lead to a common but interesting phenomenon. *R* might say “Banks are interesting!” and *S* would agree, as *S* could understand how one might find banks (as in where you deposit money) interesting.

The interesting point is where *S* would say: “Yeah, they store so much money”. *R*: “Money? At the river?”. And this is the tipping point. *R* realizes that the property of banks described by *S* is inapplicable to the object referenced by *R*. Both parties realize they have apparently been talking about different objects. They now have to adjust their object name / identifier because they have been made aware of the existence of a another similarly named object.

Now, *S*: “Yeah, you know? The banks that store money.” *R*: “Oh there banks are that store money? I thought banks were the sides of a river..” *S*: “Well, these are called banks.” Now the names have changed to “river bank” and “money bank”.

Semantic Meaning

Identifiers – like words – are nice because they partly describe what the thing is. This makes it easy for us humans to remember them. However, this may lead to problems if the meaning ever changes.

Consider the example of two Michael Jacksons. To distinguish them, one may be called “the Michael Jackson that is at Remix” and the other one “Michael Jackson, the black one”. But Michael Jackson, the musician, transitioned into white. The person stayed the same but he is not black anymore.

Meanings change, so names are not a good fit for identifiers. Only if we decouple the semantics from the id, there is nothing to misinterpret. Numbers, UUIDs, or ULIDs are a good fit.

Data

When we as humans talk about numbers or other forms of data, we usually do not reference them via a name but just write down the number we mean. There is *basically* no ambiguity when talking about numbers but more so the interpretation of them. 42 and 42 refer to the same object, the number 42. This is a different kind of object reference that is identified by itself.

Relating Things

...

Mathematical Description

Upfront there is a distinction to make between the theoretical model – the *Everything data model* (EDM) – and the implementation – the *Everything database*.

Objects, Properties, And Structures

The EDM works with objects, states facts, and places them in relation to each other. For that it presupposes a set O_A , the *set of all abstract objects*. This set contains all the things that are not identified by their structure but by themselves. For example, the sun is such an abstract object; but the equation $2 = 2$ is not because it is identified by its structure which is an equality expression (which references other objects). More examples of abstract objects may include people, trees, tables, etc.

A *property* is two-tuple of objects. The first entry (called the *tag*) represents the inherent field stated about. The second (and final) entry of the tuple represents the *value* of the property which is also an object. The *set of all properties* P is defined by $P := O \times O$. Properties describe objects.

A structure is a set of properties that identifies the structure itself. The *set of all structures* O_S is defined as:

$$O_S := \{S \mid S \subseteq P \wedge |S| \neq \infty\}$$

The *set of all objects* O is recursively defined as:

$$O := O_A \cup O_S$$

The motivation behind structures lies in the fact that without them, you would need to use an abstract object (id) and define the properties on it. However, since the structure (the object you are trying to model) is defined solely by its properties, another person trying to describe the same structure might end up with a different abstract object (id), although the properties are identical. Because to the database these are different objects, the introduction of structures is of great need.

Examples for objects

Let $a, b, c \in O_A$.

- $a \in O$
- $\{\} \in O$
- $\{(a, b), (b, c)\} \in O$
- $\{(a, \{(c, a), \{(b, \{\})\})\})\} \in O$

Associations

An *association* is a three-tuple of objects. Associations are facts. They connect objects with properties.

- The first tuple entry is nicknamed *target*. It represents the object in question, the object stated about in the association.
- The second tuple entry is nicknamed *tag*. It represents the “property” of the target stated about in this association.
- The third (and final) tuple entry is nicknamed *value*. It represents the value the property has on the target.

The set of all associations is defined as follows: $A := O \times O \times O$. (a, b, c) is an association iff $(a, b, c) \in A$.

Builtin Objects

The EDM is a self-restricting model. This will become apparent when defining the database in the following section. But for that it needs some builtin anchors such that it can interface with the implementation of the database which does the actual validation.

Therefore, we define the following set of builtin objects. The M stands for “meta” but does not have any role beyond that.

$$M_{\text{tag}}, M_{\text{unique}}, M_{\text{inferred}}, M_{\text{requires}}, \\ M_{\text{requires.not}}, M_{\text{requires.or}}, M_{\text{requires.or.not}}, M_{\text{requires.value}} : \in O$$

All these objects are pairwise distinct. Their behavior is explained in the constraints section.

Databases

A database (also called knowledge base) D is a finite set of associations ($D \subseteq A \wedge |D| \neq \infty$) that satisfies constraints (1) to (9).

General Definitions

- A *value* is an object. The alias shall denote that the object in question is used as the third member in an association.
- An object t is a *tag* iff $\exists v \in O : (t, M_{\text{tag}}, v) \in D$.
- An object a is *tagged* with an object b and a value c iff $(a, b, c) \in D$.
- An association (o, t, v) *exists* (in a database D) iff $(o, t, v) \in D$.
- $v \in O$ is a computed value of $f \in O$ and $o \in O$ iff $v \in \text{Compute}(f, o)$.
- A tag t is *applicable* to $o \in O$ iff $\text{Match}(o, t)$ is true.

$\text{Match}(o, t)$ and $\text{Compute}(f, v)$ will be defined later.

Deducted Associations

For a given database D , the *set of all deducted associations* D° is defined as:

$$D^\circ := \{(o, t, v) \in A \mid \text{Match}(o, t) \wedge (\exists f \in O : (t, M_{\text{inferred}}, f) \in D \wedge v \in \text{Compute}(f, o))\}$$

Conceptual Associations

For a given database D , the *set of all conceptual associations* $D^\diamond$ is defined as:

$$D^\diamond := D \cup D^\circ$$

With just a few steps, it can be shown that association in D are not deducted.

Let $o, t, v \in O$.

$$\begin{aligned} & (o, t, v) \in D \wedge (o, t, v) \in D^\circ \\ \stackrel{\text{Def. } D^\circ}{\implies} & (o, t, v) \in D \wedge (\exists f \in O : (t, M_{\text{inferred}}, f) \in D \wedge \text{Match}(o, t) \wedge \text{Compute}(f, o) = v) \\ \stackrel{(6) \text{ alt}}{\implies} & (\nexists f \in O : (t, M_{\text{inferred}}, f) \in D) \\ & \wedge (\exists f \in O : (t, M_{\text{inferred}}, f) \in D \wedge \text{Match}(o, t) \wedge \text{Compute}(f, o) = v) \\ \implies & \text{false} \end{aligned}$$

$$\therefore D \cap D^\circ = \emptyset$$

Constraints

- (1) Every tag, used in an association with a value v , must be tagged with M_{tag} and a value x , t must be applicable to o , and v must be tagged with x . Here is the formal constraint:

$$\forall(o, t, v) \in D : \text{Match}(o, t) \wedge (\exists x, y \in O : (t, M_{\text{tag}}, x) \in D \wedge (v, x, y) \in D^\circ)$$

The motivation behind this constraint is that you cannot arbitrarily tag objects with other objects. You have to declare the tag object as a tag. By doing this, you also constraint the objects that can be used as values in the association by defining a “type”. This type is a tag and only objects can be used as values in this association that are tagged with this type.

- (2) Tags tagged with M_{unique} enforce uniqueness on the value of associations they are involved in as a tag for a given object. Formally:

$$\begin{aligned} & \forall t \in O : ((\exists u \in O : (t, M_{\text{unique}}, u) \in D) \\ & \implies (\forall o, x, y \in O : ((o, t, x) \in D \wedge (o, t, y) \in D \implies x = y))) \end{aligned}$$

Motivation: sometimes you want a tag to be applicable maximum once per object.

- (3) M_{tag} is unique. There can only be one type describing the values of a tag. Formally:

$$\exists v \in O : (M_{\text{tag}}, M_{\text{unique}}, v) \in D$$

The motivation behind this constraint is that if you could tag a tag multiple times with M_{tag} , there would be multiple tag objects for a value to validate against. The model would need to define the validation behavior for this case but does not.

(4)

(5)

- (6) The tag M_{inferred} declares a tag as inferred. Inferred tags cannot be used explicitly in associations. However, virtually they compute the association value if the target matches the constraint. Formally:

$$\forall t \in O : ((\exists f \in O : (t, M_{\text{inferred}}, f) \in D) \implies (\nexists o, v \in O : (o, t, v) \in D))$$

This implies (by contraposition of the implication in parentheses) that if there is an association in D , the tag of that association is not inferred. Formally ((6) alt):

$$\forall t \in O : ((\exists o, v \in O : (o, t, v) \in D) \implies (\nexists f \in O : (t, M_{\text{inferred}}, f) \in D))$$

Match And Applicability

The motivation behind $\text{Match}(o, t)$ is to determine if tag t is applicable to the object o .

- To make t require that o has a different tag t' , do $(t, M_{\text{requires}}, t') \in D$. Multiple tags can be required.
- To make t require that o does not have t' , do $(t, M_{\text{requires.not}}, t') \in D$. Multiple tags can be excluded.
- To make t require that o has at least one of some tags $t_1, \dots, t_n$, do $\forall 1 \leq i \leq n : (t, M_{\text{requires.or}}, t_i) \in D$. However, if there are no $M_{\text{requires.or}}$ s, then this property should be ignored.
- To make t require that o has not every one of tags $t_1, \dots, t_n$, do: $\forall 1 \leq i \leq n : (t, M_{\text{requires.or.not}}, t_i) \in D$. However, if there are no $M_{\text{requires.or.not}}$ s, then this property should be ignored.
- If there was a value specified with

These properties formalize into this definition:

$$\begin{aligned} \text{Match}(o, t) : \iff \exists v \in O : ((t, M_{\text{required.value}}, v) \in D \implies \text{MatchValue}(o, t, v)) \\ \wedge ((t, M_{\text{required.value}}, v) \notin D \implies \text{MatchFree}(o, t)) \end{aligned}$$

$$\begin{aligned} \text{MatchFree}(o, t) : \iff & \bigwedge_{\substack{t' \in O \\ (t, M_{\text{requires}}, t') \in D}} (\exists v \in O : (o, t', v) \in D^\diamond) \\ & \wedge \bigwedge_{\substack{t' \in O \\ (t, M_{\text{requires.not}}, t') \in D}} (\nexists v \in O : (o, t', v) \in D^\diamond) \\ & \wedge \left(\bigvee_{\substack{t' \in O \\ (t, M_{\text{requires.or}}, t') \in D}} (\exists v \in O : (o, t', v) \in D^\diamond) \vee (\nexists t' \in O : (t, M_{\text{requires.or}}, t') \in D) \right) \\ & \wedge \left(\bigvee_{\substack{t' \in O \\ (t, M_{\text{requires.or.not}}, t') \in D}} (\nexists v \in O : (o, t', v) \in D^\diamond) \vee (\nexists t' \in O : (t, M_{\text{requires.or.not}}, t') \in D) \right) \end{aligned}$$

$$\begin{aligned} \text{MatchValue}(o, t, v) : \iff & \wedge \bigwedge_{\substack{t' \in O \\ (t, M_{\text{requires}}, t') \in D}} (o, t', v) \in D^\diamond \\ & \wedge \bigwedge_{\substack{t' \in O \\ (t, M_{\text{requires.not}}, t') \in D}} (o, t', v) \notin D^\diamond \\ & \wedge \left(\bigvee_{\substack{t' \in O \\ (t, M_{\text{requires.or}}, t') \in D}} (o, t', v) \in D^\diamond \vee (\nexists t' \in O : (t, M_{\text{requires.or}}, t') \in D) \right) \\ & \wedge \left(\bigvee_{\substack{t' \in O \\ (t, M_{\text{requires.or.not}}, t') \in D}} (o, t', v) \notin D^\diamond \vee (\nexists t' \in O : (t, M_{\text{requires.or.not}}, t') \in D) \right) \end{aligned}$$

Computation

The EDM

Consequences

From constraint (3) follows that there are no empty databases.

Minimal Database

$$\begin{aligned} & \{ (M_{\text{tag}}, M_{\text{tag}}, M_{\text{tag}}), (M_{\text{tag}}, M_{\text{unique}}, 0), \\ & (M_{\text{unique}}, M_{\text{tag}}, M_{\text{object}}), (M_{\text{unique}}, M_{\text{requires}}, M_{\text{tag}}), (M_{\text{unique}}, M_{\text{requires.not}}, M_{\text{inferred}}), \\ & (M_{\text{object}}, M_{\text{tag}}, M_{\text{object}}), (M_{\text{object}}, M_{\text{inferred}}, 0), \\ & (M_{\text{inferred}}, M_{\text{tag}}, M_{\text{object}}), (M_{\text{inferred}}, M_{\text{requires}}, M_{\text{tag}}), \\ & (M_{\text{requires}}, M_{\text{tag}}, M_{\text{tag}}), (M_{\text{requires}}, M_{\text{requires}}, M_{\text{tag}}), \\ & (M_{\text{requires.not}}, M_{\text{tag}}, M_{\text{tag}}), (M_{\text{requires.not}}, M_{\text{requires}}, M_{\text{tag}}), \\ & (M_{\text{requires.or}}, M_{\text{tag}}, M_{\text{tag}}), (M_{\text{requires.or}}, M_{\text{requires}}, M_{\text{tag}}), \\ & (M_{\text{requires.or.not}}, M_{\text{tag}}, M_{\text{tag}}), (M_{\text{requires.or.not}}, M_{\text{requires}}, M_{\text{tag}}), \} \end{aligned}$$